LivAna

Backend Architecture Documentation

Zero-Cost Scalable System · Up to 10,000 Users · v2.0

Stack Spring Boot + PostgreSQL	**Infra Cost** ₹0 / month
Target Scale Up to 10,000 users	**Pattern** Modular Monolith

Revised: April 2026

1. Introduction

LivAna is a multi-service student platform that aggregates PG accommodations, mess services, and grocery stores into a unified application. The system is designed to be cost-efficient (₹0 infrastructure) while maintaining scalability up to 10,000 users.

The architecture follows a modular monolith approach using Spring Boot — simple to develop and deploy, while allowing clean extraction into microservices in the future if required.

NOTE: This document (v2.0) incorporates architectural refinements including the replacement of Redis with Valkey, cursor-based pagination, API rate limiting, and a formal disaster recovery strategy.

2. System Architecture Overview

2.1 Core Stack

Component	Technology	Rationale
Backend	Spring Boot (Java 21)	Production-grade, familiar ecosystem, strong ORM support
Database	PostgreSQL 16	Relational integrity, JSONB support, free self-hosted
Cache Layer	Valkey 7.x	Open-source Redis fork (Linux Foundation), drop-in compatible, no license risk
Object Storage	Cloudflare R2	10 GB free, zero egress fees — best free storage option available
Authentication	Google OAuth 2.0 + JWT	Eliminates password storage liability; familiar to students
Push Notifications	Firebase Cloud Messaging	Free tier covers millions of messages/month
Maps	Leaflet.js + OpenStreetMap	Fully open-source, no API key required, no usage limits
Hosting	Oracle Cloud Free Tier (ARM VM)	4 OCPUs + 24 GB RAM free permanently — best free VM available

2.2 Why Valkey over Redis

Redis changed its license in 2024 to the Server Side Public License (SSPL) and Business Source License (BSL), which restricts commercial use and cloud hosting. Valkey is a Linux Foundation fork of Redis 7.2 created in direct response to this change, with backing from AWS, Google, Oracle, and Alibaba. It is:

- 100% API-compatible with Redis — no code changes required in Spring Boot (use Lettuce or Jedis as-is)
- Licensed under BSD 3-Clause — permanently free for any use
- Actively maintained with long-term foundation support
- Available as a drop-in replacement on Oracle Linux / Ubuntu

RECOMMENDATION: Install Valkey on your Oracle VM: sudo apt install valkey (Ubuntu 24.04+) or build from source at valkey.io. Your Spring Boot Redis config (spring.data.redis.*) works unchanged.

2.3 Architecture Flow

The following describes the end-to-end request lifecycle:

1. Client (Android/iOS/Web) sends an HTTP/S request to the Spring Boot backend
2. Spring Security validates the JWT token and enforces role-based access
3. Rate limiter (Bucket4j + Valkey) checks request quota per user/IP
4. Backend checks Valkey cache for the requested resource
5. On cache miss: query PostgreSQL, store result in Valkey with a TTL, return response
6. On cache hit: return cached result directly — no DB query
7. Media (images) are served directly from Cloudflare R2 URLs — never proxied through backend
8. FCM push notification dispatched asynchronously (non-blocking) for order/alert events

IMPORTANT: Media is never proxied through your backend. Always store only the R2 public URL in PostgreSQL and let clients fetch media directly. This keeps your VM bandwidth usage near zero.

3. Database Design

3.1 Core Tables

Table	Key Fields	Notes
users	id, email, google_id, role, phone, created_at	Roles: STUDENT, BUSINESS_OWNER, ADMIN
businesses	id, owner_id (FK), type, name, address, location (PostGIS point), rating, is_open	type: PG | MESS | GROCERY
listings	id, business_id (FK), title, price, description, images (text[]), city, is_active	Reused across PG rooms, mess plans, grocery items
orders	id, student_id (FK), business_id (FK), status, total_amount, created_at, updated_at	Status enum: PENDING, CONFIRMED, PREPARING, READY, DELIVERED, CANCELLED

Table	Key Fields	Notes
order_items	id, order_id (FK), listing_id (FK), quantity, price_at_order	Snapshot price prevents issues if listing price changes
reviews	id, student_id (FK), business_id (FK), rating, comment, created_at	rating: 1–5 integer
notifications	id, user_id (FK), type, payload (JSONB), is_read, created_at	Stored for in-app notification history

3.2 Indexing Strategy

Indexes are the single most impactful free performance optimization. Apply the following:

Table	Index	Purpose
listings	CREATE INDEX ON listings (city, price, is_active)	Powers the primary search/filter query
listings	CREATE INDEX ON listings (business_id)	Fast lookup of all items in a business
orders	CREATE INDEX ON orders (student_id, created_at DESC)	Student order history — most common query
orders	CREATE INDEX ON orders (business_id, status)	Business dashboard order management
reviews	CREATE INDEX ON reviews (business_id)	Rating aggregation per business
businesses	CREATE INDEX ON businesses (type, city)	Browse by category and city

3.3 Pagination — Cursor-Based (Keyset)

OFFSET-based pagination (LIMIT x OFFSET y) becomes extremely slow beyond page 30–40 on large tables because PostgreSQL must scan and discard all preceding rows. Use cursor-based pagination from the start.

CRITICAL: OFFSET pagination on a 50,000-row listings table at page 50 (OFFSET 1000) forces PostgreSQL to read and throw away 1,000 rows every time. Cursor pagination reads only what it returns.

Implementation in Spring Boot:

```
-- Instead of:
SELECT * FROM listings ORDER BY id LIMIT 20 OFFSET 200;
-- Use keyset pagination:
SELECT * FROM listings WHERE id > :lastSeenId
  AND city = :city AND is_active = true
```

```
ORDER BY id ASC LIMIT 20;
```

Return the last item's id as a nextCursor token in the API response. The frontend sends this back as a query parameter for the next page.

4. Caching Strategy — Valkey

4.1 Cache-Aside Pattern

9. Incoming request arrives for a resource (e.g., listings in Dehradun)
10. Backend checks Valkey: GET listings:dehradun:page1
11. Cache HIT → return immediately, no DB query
12. Cache MISS → query PostgreSQL → serialize result → SET listings:dehradun:page1 EX 300 → return result

IMPORTANT: Always set an explicit TTL on every cached key. Never cache without expiry — stale data is harder to debug than a slow query.

4.2 Cache Key Strategy & TTLs

Data Type	Cache Key Pattern	TTL	Invalidation Trigger
Business listings	listings:{city}:{type}:{cursor}	5 min	Any listing update in that city/type
Individual business	business:{id}	10 min	Business profile update
Menu items	menu:{business_id}	10 min	Owner adds/removes/toggles item
Search results	search:{query_hash}	3 min	TTL expiry only (acceptable staleness)
User session / JWT	session:{user_id}	JWT expiry	Logout or token revocation
Order status	order:{id}:status	30 sec	Any order status change
Business rating	rating:{business_id}	30 min	New review submitted

4.3 Cache Warm-Up on VM Restart

When the Oracle VM reboots, Valkey starts empty. Without warm-up, the first wave of requests hammers the database simultaneously — a thundering herd problem. Mitigate with:

- A Spring Boot ApplicationReadyEvent listener that pre-populates top listings for major cities on startup

- Staggered TTLs: add a small random jitter (e.g., TTL ± 30 seconds) so all keys do not expire at the same moment
- A lazy warm-up approach: first request per key hits DB, subsequent requests hit cache — acceptable for low-traffic periods post-restart

5. API Rate Limiting

5.1 Why It Is Needed

Without rate limiting, a single misbehaving client (or a bug causing retry loops in the app) can saturate your single Oracle VM and bring down the entire platform for all users.

5.2 Implementation — Bucket4j + Valkey

Bucket4j is a production-grade Java rate limiting library that integrates natively with Valkey/Redis as a distributed token bucket store. It requires zero additional infrastructure.

- Add to pom.xml: bucket4j-core + bucket4j-redis (Valkey is fully compatible)
- Implement a Spring HandlerInterceptor or filter that resolves a rate limit key per request
- Key strategy: prefer user ID (authenticated) or IP address (unauthenticated)

5.3 Rate Limit Configuration

Endpoint Category	Limit	Window	Notes
General API (authenticated)	200 requests	1 minute	Comfortable for normal app usage
Search / Browse listings	60 requests	1 minute	Prevent scraping
Order placement	10 requests	1 minute	Prevent accidental duplicate orders
Authentication / Login	10 attempts	15 minutes	Brute-force protection
Image upload	20 uploads	1 hour	Limit R2 write operations
Unauthenticated endpoints	30 requests	1 minute	Per IP address

NOTE: Return HTTP 429 Too Many Requests with a Retry-After header when a limit is breached. The Android/iOS app should respect this header and back off gracefully.

6. Authentication & Security

6.1 Authentication Flow

13. User taps 'Sign in with Google' in the mobile/web app
14. App receives Google ID token via Google Sign-In SDK
15. App sends ID token to POST /api/auth/google
16. Backend validates token against Google's public keys (no Google API call needed — use google-auth-library)
17. On first login: create user record in PostgreSQL
18. Backend issues a signed JWT (access token: 15 min) + refresh token (stored in Valkey, 7 days)
19. Client stores JWT in secure storage; includes it as Authorization: Bearer <token> on every request
20. Refresh flow: client calls POST /api/auth/refresh with the refresh token to get a new JWT

6.2 Role-Based Access Control

Role	Permissions	Notes
STUDENT	Browse listings, place orders, write reviews, manage own profile	Default role on registration
BUSINESS_OWNER	All student permissions + manage own business, menu, orders	Assigned after business onboarding
ADMIN	Full access including user management, dispute resolution	Manual assignment only

6.3 Security Hardening Checklist

- HTTPS enforced via Cloudflare proxy (free) — never expose raw HTTP
- JWT signed with RS256 (asymmetric) — rotation possible without invalidating all sessions
- Refresh tokens stored in Valkey with TTL — revocation is instant (DELETE key)
- Spring Security method-level annotations (@PreAuthorize) on all sensitive endpoints
- Input validation via Jakarta Bean Validation (@Valid) on all request DTOs
- SQL injection impossible via Spring Data JPA parameterized queries — never use string concatenation in queries
- Cloudflare WAF (free tier) provides basic DDoS and bot protection in front of your VM
- Never log JWT tokens, passwords, or payment tokens — log only user IDs

7. Media Storage — Cloudflare R2

7.1 Strategy

- All images (business photos, menu items, PG room photos, user avatars) stored in Cloudflare R2
- Database stores only the public R2 URL as a text string — never binary data

- Clients fetch images directly from R2 — your backend VM never proxies image traffic
- R2 free tier: 10 GB storage + 10 million Class A operations + 1 million Class B operations per month

IMPORTANT: This is critical. If you proxy images through your backend, a single user loading a restaurant menu with 10 photos generates 10 backend requests and significant bandwidth. R2 direct delivery eliminates this entirely.

7.2 Upload Flow

21. Client requests a pre-signed upload URL from POST /api/media/upload-url
22. Backend generates a pre-signed R2 PUT URL (valid for 5 minutes) via the R2 S3-compatible API
23. Client uploads the image directly to R2 using the pre-signed URL — backend is not in the data path
24. Client confirms upload success to backend: POST /api/media/confirm with the R2 object key
25. Backend stores the public R2 URL in the relevant database record

7.3 Image Guidelines

Type	Max Size	Format	Recommended Dimensions
Business cover photo	2 MB	JPEG/WebP	1280 × 720 px
Menu item photo	1 MB	JPEG/WebP	800 × 600 px
PG room photo	2 MB	JPEG/WebP	1280 × 720 px
User avatar	500 KB	JPEG/PNG	400 × 400 px

Enforce size limits at the pre-signed URL generation step — reject requests exceeding the limit before the upload is attempted.

8. Real-Time Features

8.1 Push Notifications — FCM

Firebase Cloud Messaging handles all real-time user-facing alerts. It is asynchronous — the backend queues the notification and FCM delivers it. This is the correct approach for the current scale.

Trigger	Recipient	Notification Content
Order placed	Business owner	New order #ORD-2214 from Arjun Mehta — ₹185
Order status updated	Student	Your order is being prepared / is ready for pickup

Trigger	Recipient	Notification Content
Order cancelled	Student + Owner	Order #ORD-2209 has been cancelled
New review received	Business owner	Anjali gave you 5 stars on Dal Makhani
Promotional message	Targeted students	Maa Ki Rasoi: Today's special — Thali at ₹99

8.2 Live Order Tracking — WebSockets (Deferred)

Spring WebSockets with STOMP can power live delivery tracking. This should not be built in v1. The conditions to introduce it are:

- Actual delivery (rider-to-door) orders exist and are flowing
- Students are complaining about not knowing delivery status
- The FCM polling approach (status refresh every 15 seconds) is insufficient

NOTE: WebSocket connections are long-lived and consume memory per active connection. At 500 concurrent delivery-tracking sessions, this is ~50 MB RAM — manageable on Oracle's 24 GB VM, but do not build it until genuinely needed.

9. Backup & Disaster Recovery

9.1 The Oracle Free Tier Risk

Oracle Cloud Free Tier instances are subject to reclamation if Oracle needs capacity, with minimal advance notice. This is the single highest risk in the architecture. A production PostgreSQL database with no backup is a critical liability.

CRITICAL: Oracle has reclaimed always-free VMs with as little as 24–48 hours notice. Do not treat Oracle as a permanent hosting solution without a backup strategy.

9.2 Backup Strategy

What	How	Frequency	Destination	Cost
PostgreSQL full dump	pg_dump | gzip → rclone sync to R2	Daily (cron 2:00 AM)	Cloudflare R2 (backups/ prefix)	₹0
PostgreSQL WAL archiving	archive_command in postgresql.conf → R2	Continuous	Cloudflare R2	₹0
Valkey RDB snapshot	BGSAVE + copy to R2	Every 6 hours	Cloudflare R2	₹0
Application config / secrets	Encrypted export to Backblaze B2	On every change	Backblaze B2 (10 GB free)	₹0

Restoration target: RTO (Recovery Time Objective) of 2 hours — set up a new Oracle VM, restore the latest pg_dump, redeploy the Spring Boot JAR from the latest GitHub release.

9.3 VM Health Monitoring

- Enable Oracle Cloud's built-in instance monitoring (CPU, memory, disk) — free
- Use Spring Boot Actuator + a free UptimeRobot account to monitor /actuator/health every 5 minutes
- UptimeRobot sends email/SMS alerts when the endpoint goes down — free for up to 50 monitors
- Keep a Dockerfile and GitHub Actions deploy workflow ready — redeployment to a fresh VM should take under 30 minutes

10. Scalability Roadmap

Phase 1 — Launch to 5,000 Users (Current Architecture)

Component	Config	Handles
Oracle VM	4 OCPUs, 24 GB RAM (ARM)	All services on one VM
Spring Boot	Single JAR, thread pool tuned	~200 concurrent requests
PostgreSQL	postgresql.conf tuned for 24 GB RAM	~5,000 active users, millions of rows
Valkey	maxmemory 4g, allkeys-lru policy	Hot data cache, sessions, rate limits
Cloudflare R2	Direct media delivery	Unlimited media bandwidth (free egress)
Cloudflare Proxy	Free tier, WAF enabled	DDoS protection, HTTPS termination

Phase 2 — 5,000 to 10,000 Users

- Promote Valkey caching from optional to mandatory — all list/search endpoints must hit cache first
- Enable PostgreSQL connection pooling via PgBouncer (install on same VM) — prevents connection exhaustion
- Add read replica: Oracle's Always Free tier permits a second VM — run a PostgreSQL streaming replica for read-heavy queries
- Profile slow queries using pg_stat_statements and add targeted indexes
- Introduce async job processing: Spring @Async or a lightweight embedded queue (PGMQ — PostgreSQL-based) for notification dispatch, image processing, and report generation

Phase 3 — Beyond 10,000 Users (Future)

- Extract high-traffic services (Orders, Notifications) as separate Spring Boot microservices
- Introduce Apache Kafka for event streaming between services
- Migrate to managed PostgreSQL on Supabase or Neon (both have generous free tiers) to eliminate self-managed DB ops
- Horizontal scaling behind a load balancer (Caddy or Nginx) with multiple Spring Boot instances
- Consider Fly.io or Railway for managed deployment — both offer free tiers competitive with Oracle

11. Cost Analysis

11.1 Monthly Infrastructure Cost — ₹0

Component	Service	Free Limit	Cost
Server	Oracle Cloud Free Tier (ARM VM)	4 OCPUs, 24 GB RAM, 200 GB disk — permanent	₹0
Database	PostgreSQL (self-hosted on Oracle VM)	Unlimited on free VM	₹0
Cache	Valkey (self-hosted on Oracle VM)	Unlimited — open source	₹0
Object Storage	Cloudflare R2	10 GB storage, zero egress fees	₹0
Maps	OpenStreetMap + Leaflet.js	No limits	₹0
Push Notifications	Firebase Cloud Messaging	Unlimited messages	₹0
DDoS / HTTPS	Cloudflare Free Proxy + WAF	Unlimited requests	₹0
Uptime Monitoring	UptimeRobot Free	50 monitors, 5-min interval	₹0
CI/CD	GitHub Actions Free	2,000 minutes/month	₹0

11.2 Variable Costs

Item	Rate	Notes
Payment gateway (Razorpay / PhonePe)	~2% per transaction	Only incurred when transactions occur — no fixed cost
SMS OTP	~₹0.12 per SMS	Minimise by making Google OAuth the primary/only login method
Cloudflare R2 overage	₹1.30 / GB storage above 10 GB	Unlikely until significant media accumulation

11.3 One-Time Costs

Item	Cost	Notes
Domain name	~₹800 / year	Namecheap or GoDaddy; .in domain available for ~₹600
Google Play Console	$25 (~₹2,100)	One-time, covers all apps on your account
Apple Developer Program	$99 (~₹8,300) / year	Required only if building an iOS app

Total monthly infrastructure cost: **₹0** (excluding variable transaction fees)

12. Conclusion & Action Items

This architecture provides a robust, secure, and genuinely zero-cost backend capable of handling LivAna's full feature set for up to 10,000 users. The key refinements in v2.0 over the original design are:

Refinement	Original	v2.0
Cache layer	Redis (SSPL/BSL licensed)	Valkey 7.x (BSD-3, Linux Foundation)
Pagination	OFFSET-based (slow at scale)	Cursor-based / keyset pagination
Rate limiting	Not addressed	Bucket4j + Valkey (per user/IP)
Backup strategy	Not addressed	Daily pg_dump + WAL archiving → R2
Cache cold start	Not addressed	Warm-up listener + TTL jitter
Security	Basic JWT mention	RS256 JWT, Cloudflare WAF, full checklist
Media upload	Vague	Pre-signed R2 URLs — backend not in data path
Monitoring	Not addressed	Actuator + UptimeRobot

Immediate Action Items

26. Replace any Redis dependency in pom.xml with Valkey-compatible Lettuce client (no change needed — Lettuce works with Valkey out of the box)
27. Set up the Oracle VM with: PostgreSQL 16, Valkey 7, Java 21, Nginx as a reverse proxy
28. Configure automated daily pg_dump to Cloudflare R2 before going live
29. Implement cursor-based pagination in all list/search Spring Data JPA repositories
30. Add Bucket4j rate limiting filter before launch — not after
31. Point a free Cloudflare proxy in front of your Oracle VM's public IP
32. Register UptimeRobot monitor for your /actuator/health endpoint

RECOMMENDATION: Ship the modular monolith. The architecture is sound. The most common failure mode for student projects is over-engineering before launch, not under-engineering. Get to 100 real users, then iterate.